

Credit Card Default Prediction

The objective of this project was to develop a credit scoring algorithm that predicts the likelihood of an individual encountering financial distress within the next two years. While working on the project we focused on mainly two parts: analyzing the variables of the dataset and cleaning the dataset, and testing different credit scoring algorithms in order to select the one that better predicts the probabilities of credit default.

Handling and Analysis of the Dataset

In this very first part of the project we loaded the dataset and started analyzing thoroughly each variable of the data set.

The dataset comprises 12 variables and 112,500 observations. The variable 'SeriousDlqin2yrs' serves as our target variable, which is binary and assumes a value of 1 when an individual has experienced a delinquency of 90 days or more. At this stage, we started investigating each variable: for each variable we plotted them graphically (both their density and frequency) and printed the summary. This preliminary exploration offered us an extensive understanding of the dataset's components and their distribution.

We observed a significant imbalance in the dataset: 104,980 observations are labeled with 'SeriousDlqin2yrs' = 0, indicating no delinquency, while only 7,520 are labeled with 'SeriousDlqin2yrs' = 1, indicating delinquency. Furthermore, we identified an anomaly in the 'Age' variable where one observation recorded an age of -1, which is implausible within the context of our analysis. Consequently, we decided to exclude this observation from the dataset.

Imputing

For each variable, we assessed the presence of missing values (NA). Our analysis revealed that the "MonthlyIncome" variable has 19.72% missing values, while the "NumberOfDependents" variable has a mere 0.026% missing. For the latter variable, due to the small percentage of data missing, we used the Median Imputation method which consisted in the substitution of the NA values with the median of the rest of the observations of "NumberOfDependents". The preference of this method over the mean imputation was due its lower sensibility to the presence of outliers.

Due to the significant number of missing values in the "MonthlyIncome" variable, we performed a graphical analysis to determine whether the missing values were correlated with any other variables in the dataset. We used a boxplot to compare variables whose observations where "MonthlyIncome" was missing against those where it was present. From these observations, it appears that missing income data is possibly associated with higher financial activity and stress, evidenced by higher debt ratios, more credit lines, higher credit utilization, and more frequent past-due payments. This might suggest that individuals whose income data is missing could either have complex income sources that are difficult to report or are experiencing financial instability.

However, these records show a very low incidence of serious delinquency then, to preserve the integrity of our dataset and avoid discarding a significant portion of the data, we chose to impute these missing entries.

We tried to predict the missing "MonthlyIncome" values using 4 alternative imputation methods: mean imputation, median imputation, KNN imputation and Iterative imputation with the use of RandomForest as estimator. We choose Random Forest as the estimator for the Iterative Imputer

since it is a versatile model capable of capturing complex relationships between features and target variables and it is robust to outliers.

To find the best imputation method for our dataset, we first copied the training dataset, dropped the missing values and we split the remaining observations into a new training set (80% of data) and test set (20%). In order to compare them effectively, we built two functions:

- `create_missing_values`: This function randomly introduces missing values into specified columns of a dataset based on a given percentage, ensuring consistency between training and testing datasets. It returns the dataset with missing values and a boolean mask indicating their locations.
- `Update_metrics`: This function updates a metrics dataframe with Mean Absolute Error (MAE) and Mean Squared Error (MSE) for the "MonthlyIncome" variable in the imputed test data. It calculates the errors based on the ground truth test data and the mask indicating missing values for this variable. It return these two metrics with the corresponding `model_name`

Two of the imputers, KNN imputer and Iterative Imputer with Random Forest, need multiple parameters to be selected before operating. For the KNN imputer, we tuned the parameters using the GridSearchCV method, which, by exhaustively searching through a specified grid of hyperparameters, allows for the selection of the optimal ones. On the other hand, the parameters of both the Random Forest estimator and the Iterative Imputer were selected manually based on the knowledge acquired through the previous analysis of the dataset due to the computational complexity of the GridSearch cross validation.

Having some outliers in our dataset, which we didn't eliminate to avoid lack of representativity of real data in the imputation, we choose MAE as the best metric for comparison, since it is more robust to them (Fig1).

	mae_4	mse_4
MeanImputer	4275.65372	2346260636.170592
	mae_4	mse_4
MedianImputer	4114.019701	2349708818.409696
	mae_4	mse_4
KNNImputer	4162.415566	2373022765.115994
	mae_4	mse_4
Iterative imputer (random forest)	3277.408541	2109485956.071268

By choosing this selection criteria, the missing values of the column MonthlyIncome are imputed using the Iterative Imputer whose estimator is random forest, as imagined given its characteristics.

Fig1: Comparison of MAE and MSE of the four imputation methods

Outliers

We next assessed the presence of outliers in the dataset, employing two methods for their identification: the first method tagged observations lying more than two standard deviations from the mean as outliers, while the second method identified outliers as observations below the 0.01 percentile and above the 0.99 percentile.

Given that outliers can distort data distribution and compromise model accuracy, we opted to remove them. Before proceeding with the removal, we conducted a visual analysis to ensure that eliminating these outliers would not omit crucial information, particularly in terms of their correlation with the target variable. The presence of significant outliers across several key financial indicators, such as debt ratio and credit utilization, which correlate with serious delinquency, suggests that retaining outliers may provide crucial insights into financial behavior and risk assessment. For this reason, to avoid deleting a large portion of the data, we decided to only delete outliers above the 99th and below the 1st percentile of our dataset as this method was less likely to discard a large portion of the dataset (for example, the first method considered as outliers the 8.61% of observations of the variable "NumberOfDependents").

Feature Engineering & Selection

After having addressed the missing data and outliers issue, we decided to create new features from the existing ones in order to provide the model with better-quality input features that could help to capture more complex patterns in the data and improve the whole performance. We added “Income for dependent” and “Delinquency ratio”, defined as the sum of the number of times the person committed delinquency over the number of open loans and lines of credit.

Furthermore, to capture eventual non-linear relationship between the variables, we created a PolynomialFeatures object which expanded the feature space present in the dataframe by including polynomial terms of the original features up to the second degree. The choice of the second degree was to prevent eventual overfitting. The new data frame consists of 67 variables which subsequently we proceeded to select using RFCEV, Recursive Feature Elimination with Cross-Validation. It is a powerful technique that trains the model on the full set of features, ranks them by relevance and proceed to eliminate the least useful features. To assess the generalization ability of this model, cross validation is used after each round of feature elimination. The choice of its use was due its ability to reduce overfitting and enhancing model prediction efficiency through the reduction of the dimensionality of the feature space. We built two RFCEV models: one uses XGBoost Classifier whereas the other uses Random Forest as an estimator. Once we had the features chosen by each model, we decided to proceed our work only with the aid of the features selected by both models which were 13.

Multicollinearity

In the subsequent phase of our analysis, we analyzed the correlations among the variables of the dataset. Addressing multicollinearity and high correlation between variables is crucial in data classification methods as it helps prevent redundancy and ensures that the predictive model remains stable and interpretable. To conduct this investigation we employed a correlation matrix heatmap. However, rather than using Pearson's correlation typically utilized in heatmaps, we opted for Spearman's method. Spearman's rank correlation is particularly adept at dealing with datasets rich in count data, like ours, because it evaluates how well the relationship between two variables can be described by a monotonic function, effectively capturing relationships that are not linear (Fig2)

After displaying the heatmap, we analyze two groups of pairs of features: pairs of features with modest correlation (between 0.4 and 0.7) and pairs of features with high correlation (over 0.7). The first subset was analyzed with the aid of scatterplots and we noticed the only pair that clearly showed multicollinearity was “DebtRatio vs. DebtRatio-NumberRealEstateLoansOrLines”. Because of that, we decided to consider that pair as part of the high correlation group. Subsequently, the features whose correlation was over 0.7 were ranked based on their relevance and we decided to drop one from each pair, being careful not to keep any combination of highly correlated features. We ended up dropping 5 features, obtaining the final feature space composed by 8 explanatory variables (Fig3).

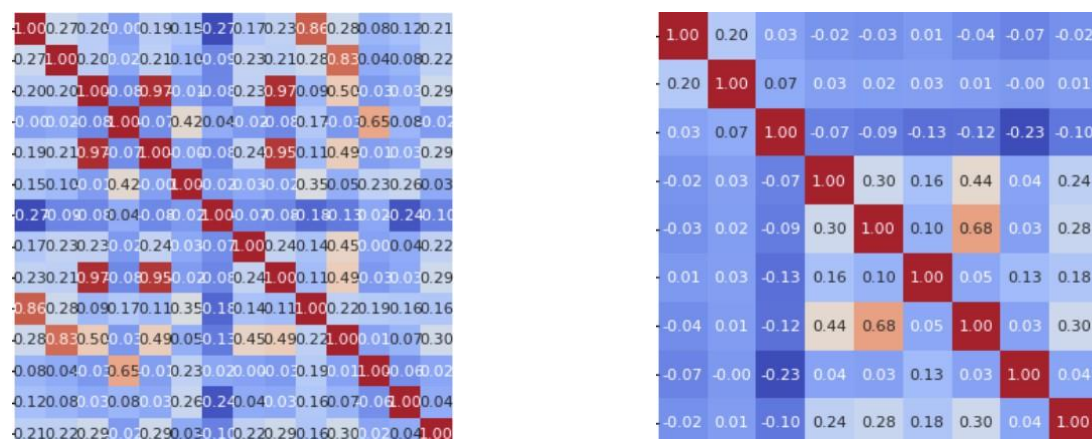


Fig2: Correlation matrix heatmap before selection based on correlation

Fig3: Final Correlation matrix heatmap

Scaling

After dealing with multicollinearity, we proceeded with scaling. The transformation of feature values to a similar scale ensures that they all have a similar magnitude, preventing certain features from dominating the learning process solely because of their larger scale, especially in models which rely on distance metrics. The choice of Standardization as the scaling method is imputable to some of the characteristics of our dataset, as the largely different scales of the variables, and we applied it separately to the training and test set to avoid information from the test set to influence the scaling of the training set, which can lead to data leakage.

Model choice and performance analysis

Once the manipulation of the dataset was completed, we decided to infer the probability of the target variable with four different models: logistic regression, a Bagging technique, using Decision Trees, and two Boosting techniques, XGBoost and Histogram-based Gradient Boosting. To boost the efficiency of XGBoost and Histogram-based Gradient Boosting, we tuned their parameters. This tuning was crucial because it allowed us to refine how these algorithms handle data, improving both their speed and accuracy. In the tuning phase, we used grid search to find the best parameters and the parameters that seemed to have more influence on our results were specifically tuned to ensure that they were optimized to enhance our results effectively.

Logistic Regression was the first model we tested, given its simplicity and efficiency, it served as our baseline model. In the Logistic Regression analysis, the model achieved an accuracy of 81.4% with a ROC AUC of 0.744, indicating good discriminatory power between classes. The low F1 score of 29.5% reflects the imbalance between precision and recall. We then plot the confusion matrix, that revealed the fact that the model succeeded in identifying true negatives but often predicts false positives, indicating potential overfitting to the majority class.

We then decided to implement the Bagging technique using decision trees as base models, our aim was to reduce variance and overfitting of the data. The results indicated that while accuracy of the Bagging model was notably high at 0.940, its overall effectiveness was limited by a lower recall of 0.130 and a ROC AUC of 0.560. This suggests that while the model was accurate in its positive predictions, it failed to identify a significant number of actual default cases, likely due to overfitting to the non-default cases. Moreover, the F1 score of 0.200 further reflects the imbalance between precision and recall, indicating that the model, despite its precision, is not adequately capturing the true default instances.

Subsequently we then decided to use XGBoost, as it is widely known for its performance in classification tasks and for its ability to scale and manage large datasets. Pre-tuning, it showed a recall of 0.706, increasing to 0.726 post-tuning, demonstrating the benefits of optimizing model parameters. However, this came with a slight trade-off in precision, which decreased from 0.186 to 0.180, with the F1 score slightly decreasing as well, suggesting a more conservative model in terms of predicting positives. The model's ability to maintain strong performance metrics, like an increase in ROC AUC from 0.758 to 0.761, demonstrates its effective discrimination between defaulters and non-defaulters.

Histogram-based gradient boosting is an iteration over traditional gradient boosting, that uses histograms to optimize splitting points in decision trees; it is widely known for its speed and efficiency. Initially, it achieved a recall of 0.723, demonstrating its capability to identify a high number of true positives, crucial for applications like credit default prediction where missing a default can be costly. Its precision, however, started at 0.180 and slightly declined to 0.176 after tuning, indicating a continued challenge in minimizing false positives. Overall the model's strengths lie in its recall capabilities, but it struggles to balance this with an improvement in precision.

Here we reported the results from the models:

Model	Accuracy	ROC AUC	F1 score	Precision	Recall
Logistic Regression	0.814	0.744	0.294	0.188	0.666
Bagging (decision tree)	0.940	0.560	0.200	0.435	0.130
XGBoost	0.803	0.758	0.293	0.186	0.706
XGBoost (tuning)	0.792	0.761	0.288	0.180	0.726
Hist gradient boosting	0.792	0.756	0.287	0.180	0.723
Hist gradient boosting (tuning)	0.787	0.759	0.284	0.176	0.726

Comparison of XGboost and Histgradientboost (after tuning):

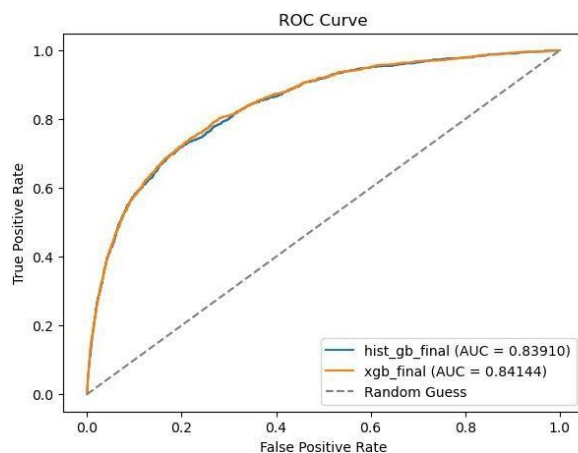


Fig4: Comparison of ROC curve of hist gradient boosting and XGBoostselection

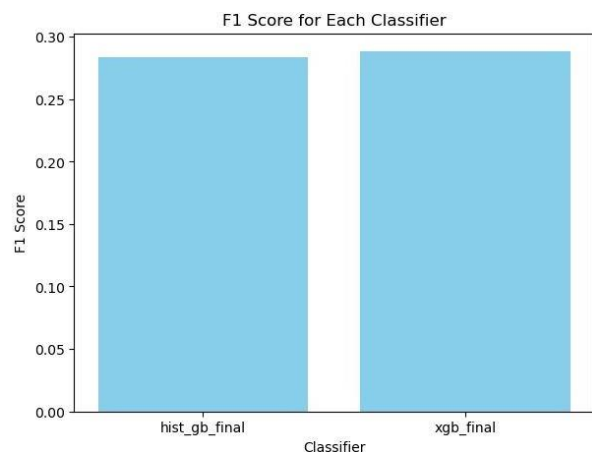


Fig5: Comparison of F1 score of hist gradient boosting and XGBoost

As final method we chose XGBoost, due to its ability to balance recall and precision effectively, crucial in financial settings where identifying actual defaults is paramount. Its enhanced recall, minimal loss in precision, and improved ROC AUC highlight its robustness and adaptability in managing large and complex datasets. Compared to other models like Logistic Regression and Histogram-based Gradient Boosting, XGBoost demonstrates better overall performance, making it particularly suited for reliably predicting credit defaults in dynamic financial environments.